COSC2731 - Tutorial 3

Tuan Phong Ngo

phong.ngo@rmit.edu.vn



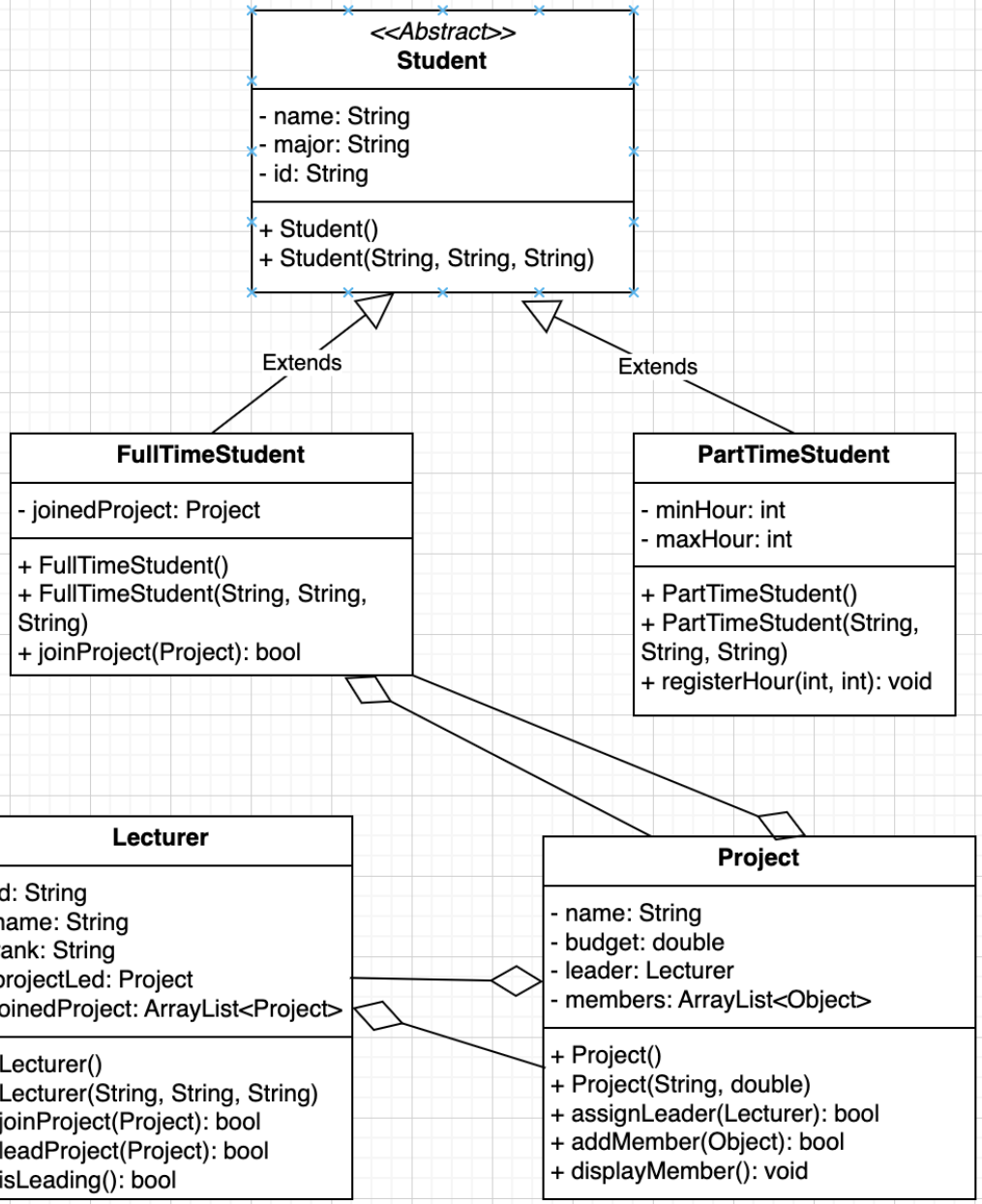
RMIT UNIVERSITY

Outline

1. Question 1
2. Question 2
3. Question 3
4. Question 4



ation: Trusted

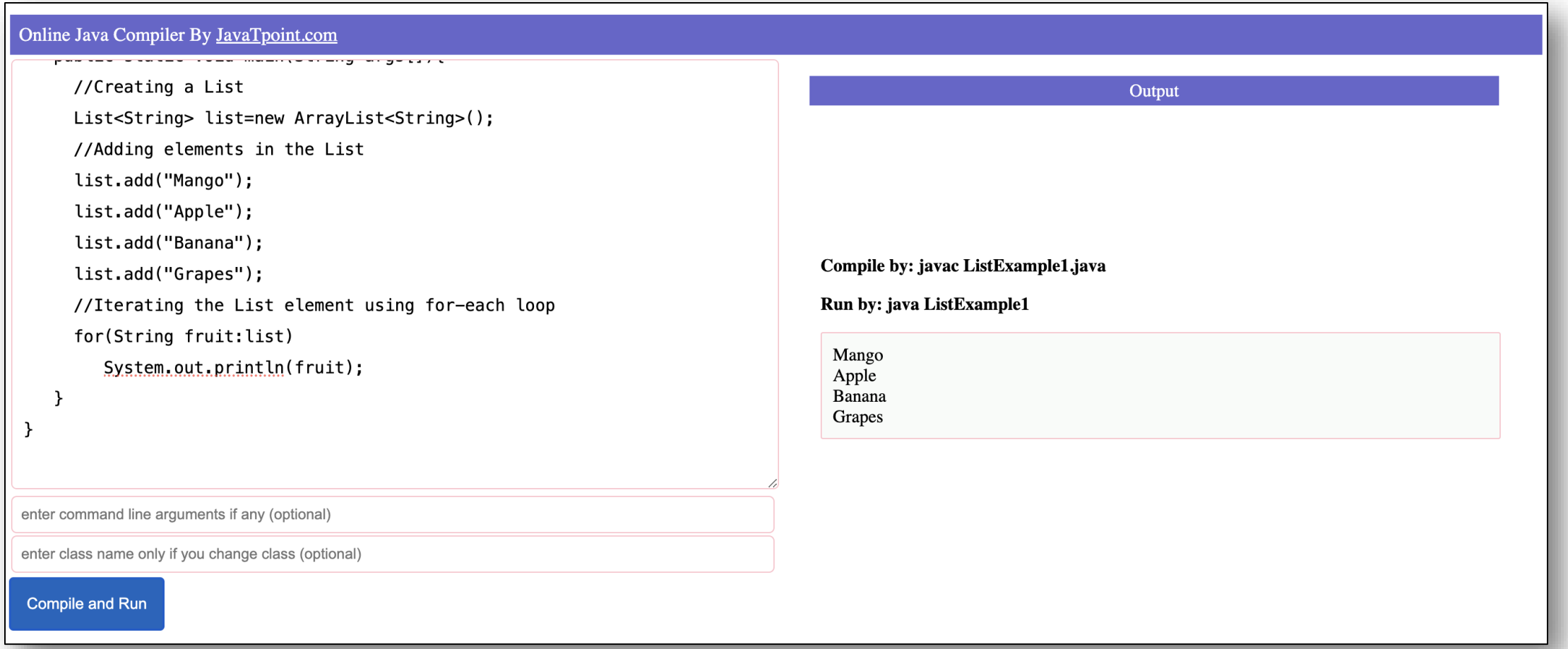


Question 1 – Use List Interface

Update the current code to use the **List interface** to manage the **members** of a project.

```
3   public class Project {
        3 usages
4       private String name;
        2 usages
5       private double budget;
        3 usages
6       private Lecturer leader;
        4 usages
7       private Researcher[] members;
        3 usages
8       private final int MAX_MEMBERS = 10;
        4 usages
9       private int memberCount = 0;
10
```

List example



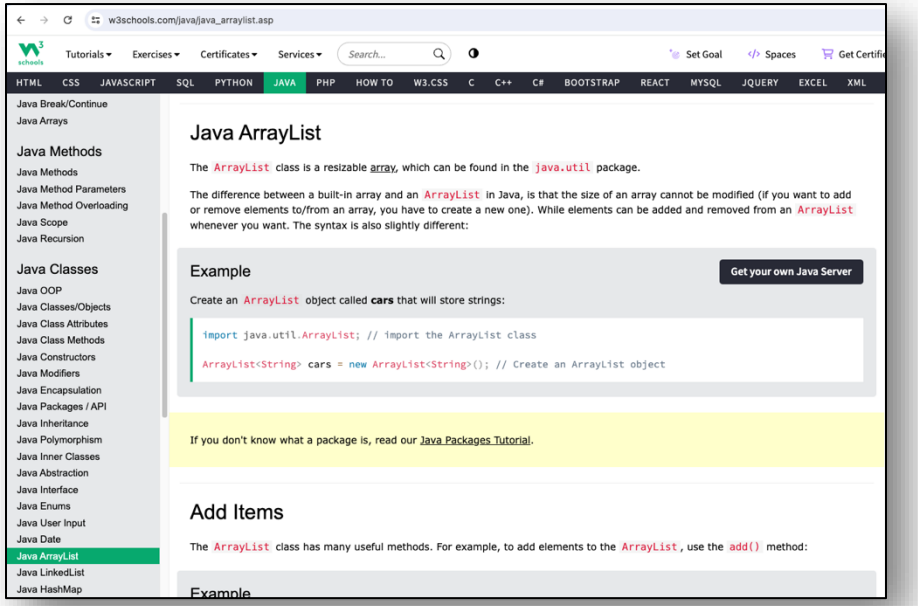
https://compiler.javatpoint.com/opr/test.jsp?filename=ListExample1
https://docs.oracle.com/javase/8/docs/api/?java/util/List.html

ation: Trusted

List interface

```
7   public class Project {
        3 usages
8       private String name;
        2 usages
9       private double budget;
        3 usages
10      private Lecturer leader;
11      //private Researcher[] members;
12      //private final int MAX_MEMBERS = 10;
13      //private int memberCount = 0;
14      // Use List interface
        7 usages
15      private List<Researcher> members;
```

- Import corresponding libraries
- List is an interface → use it together with ArrayList or LinkedList



https://www.w3schools.com/java/java_arraylist.asp

Question 2 – Set interface for LeaderSet

- Create a set of leaders (a leader is a lecturer who is leading a project) **LeaderSet** containing operations:
  - Check whether a lecturer is leading a project or not
  - Add a lecturer to the set
  - Remove a lecturer from the set
- Two lecturer instances are considered **equal** if they have the same employee id.

hashCode() and equals() contract for contains() of Set

hashCode() and equals() contract is that:

- two objects are equal(by using equals() method), they must have the same hashCode().
- If two objects have same hash code, they may be not equal.

```
import java.util.HashSet;

class Dog{
        String color;

        public Dog(String s){
                color = s;
        }
}

public class SetAndHashCode {
        public static void main(String[] args) {
                HashSet<Dog> dogSet = new HashSet<Dog>();
                dogSet.add(new Dog("white"));
                dogSet.add(new Dog("white"));

                System.out.println("We have " + dogSet.size() + " white dogs!");

                if(dogSet.contains(new Dog("white"))){
                        System.out.println("We have a white dog!");
                }else{
                        System.out.println("No white dog!");
                }
        }
}
```

```
class Dog{
        String color;

        public Dog(String s){
                color = s;
        }

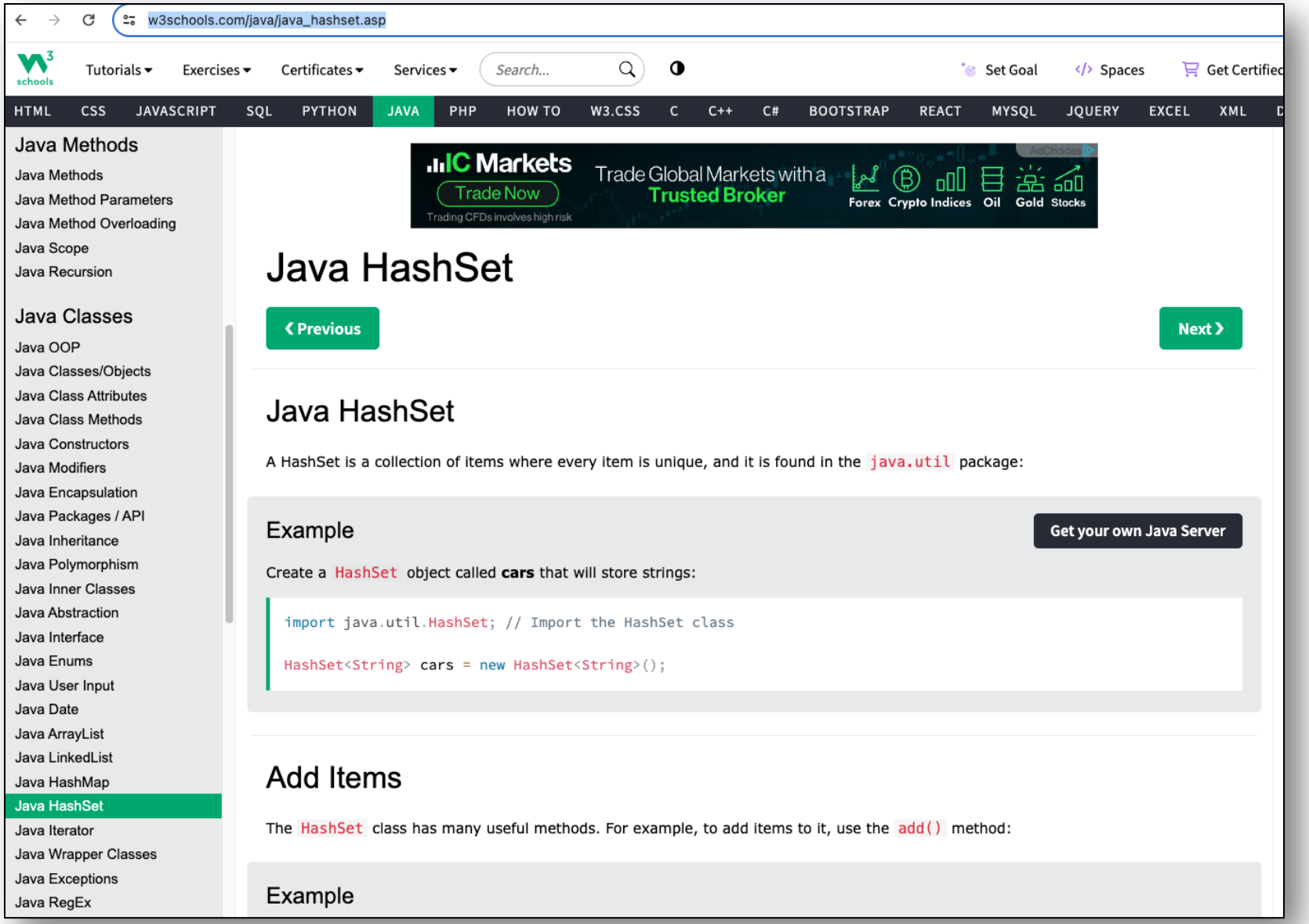
        //overridden method, has to be exactly the same like the following
        public boolean equals(Object obj) {
                if (!(obj instanceof Dog))
                        return false;
                if (obj == this)
                        return true;
                return this.color.equals(((Dog) obj).color);
        }

        public int hashCode(){
                return color.length();//for simplicity reason
        }
}
```

We have 2 white dogs!

No white dog!

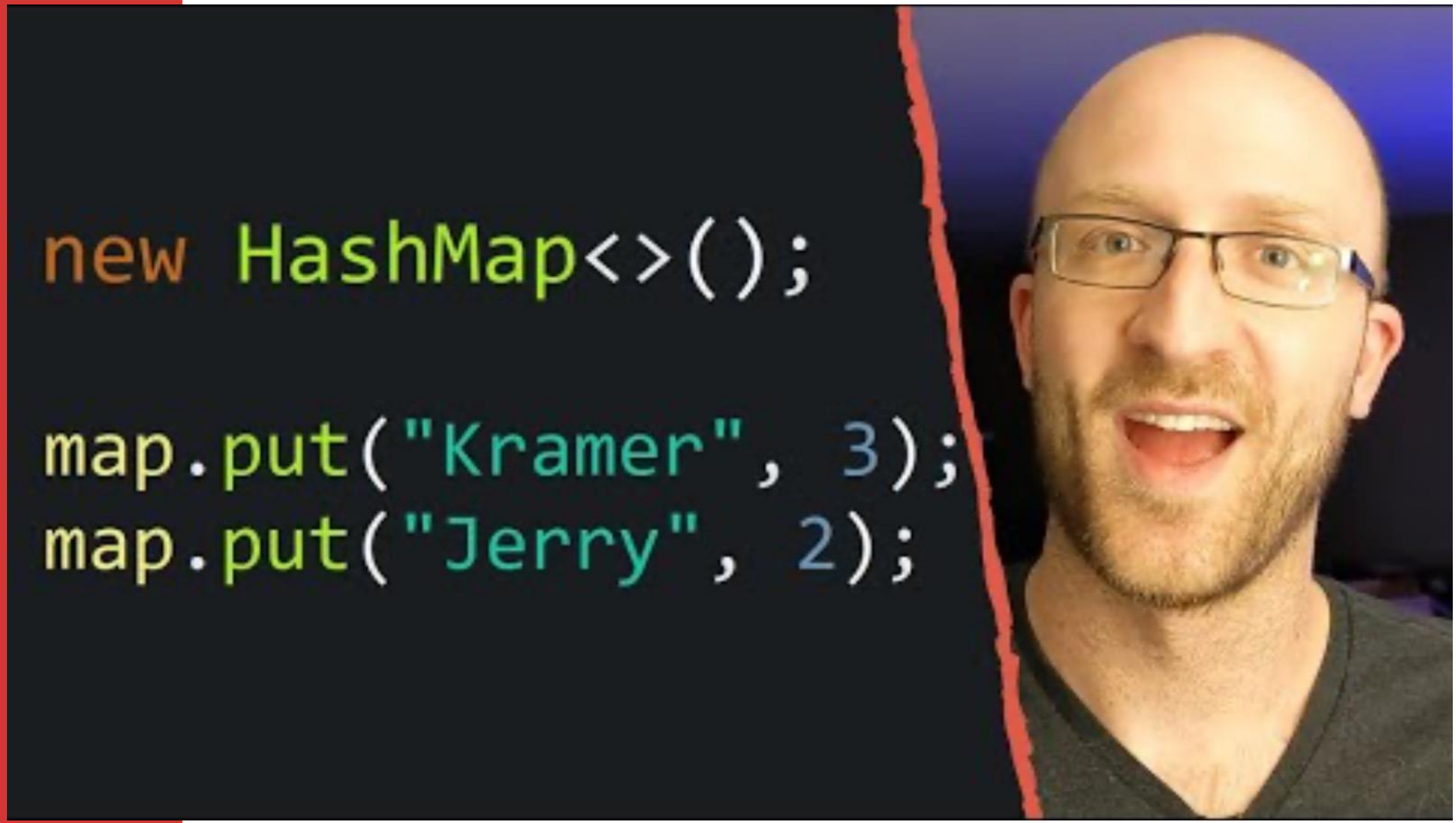
contains(o) = true if and only if this set contains an element e such that (**o==null ? e==null : o.equals(e)**)



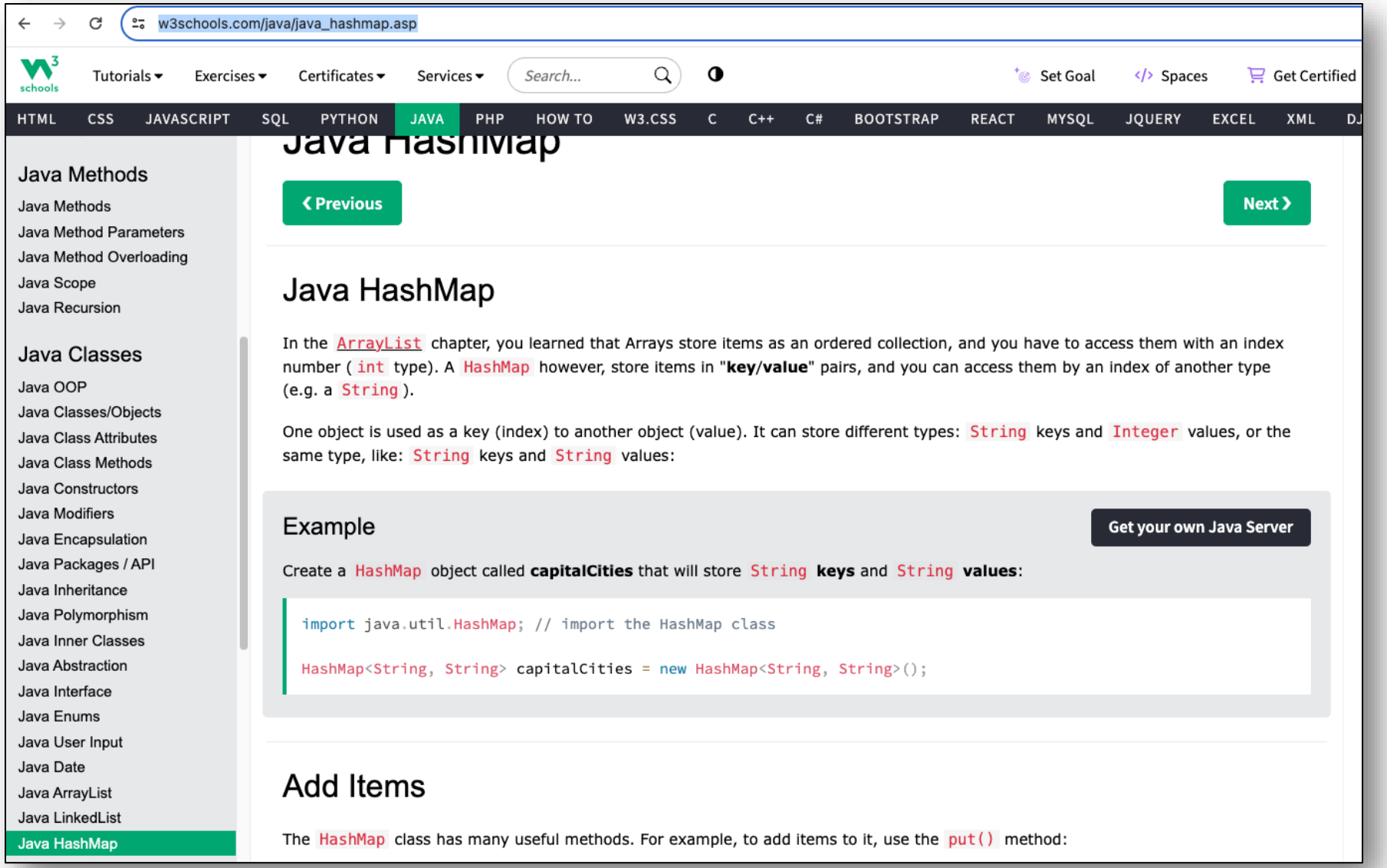
https://www.w3schools.com/java/java_hashset.asp

Question 3 – Map interface for ProjectMember

- Implement a **ProjectMember** map such that:
  - Given a Researcher, return a **list of projects** this person is doing (or null if this person is not a member of any project)
  - **Add** a project to a Researcher's list of project
  - **Remove** a project from a a Researcher's list of project



https://www.youtube.com/watch?v=H62Jfv1DJlU



https://www.w3schools.com/java/java_hashmap.asp

Question 4 – Comparator interface

- Given a Researcher, return a list of projects this person is doing **in reversed order of budget** (i.e., the project with the highest budget is positioned first in the list)



https://www.geeksforgeeks.org/comparable-vs-comparator-in-java/

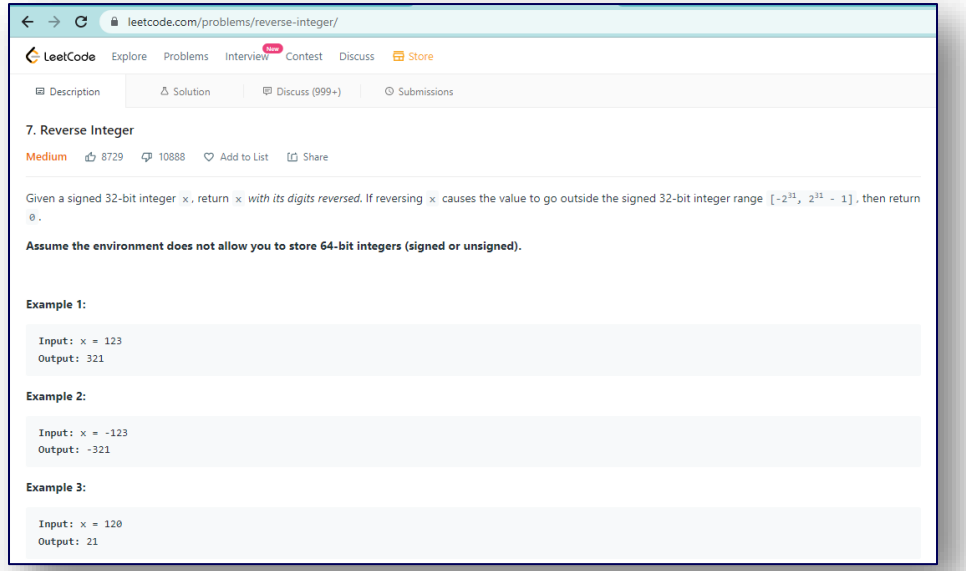
Learning at home

Review lectures

Finish this week tutorial

Do Early Feedback quiz if needed

Optional: Solve some easy/medium problems at https://leetcode.com/problemset/all/



Have fun coding &
See you next week

RMIT UNIVERSITY